CS1632: Defects

Wonsun Ahn

Defects and Enhancements

Importance of understanding implicit requirements

Review: Defects and Failures

- Defect: A software flaw that can cause a failure
- Failure: When *observed behavior ≠ expected behavior*

- How can we know *expected behavior*?
  - One word: *Requirements*

Defects vs Enhancements

- A software QA team has two jobs:
  - Find and report *defects*
  - Find and suggest *enhancements*

- What's in common between *defects* and *enhancements*?
  - Both involve modifications to software that can improve software quality
- What's the difference?
  - *Defect*: A flaw that can cause a violation of the requirements
  - *Enhancement*: A proposed improvement to existing requirements

Differentiating Defects vs Enhancements

- Differentiating is important: determines who pays
  - *Defect*: developer fixes it at no additional cost to the customer
  - *Enhancement*: customer pays developer for the added improvement

- Differentiating sounds easy enough!
  - If software violates pre-existing requirements → defect
  - If software doesn't violate pre-existing requirements → enhancement

- But sometimes differentiating the two is surprisingly hard
  - Mainly due to *implicit requirements*

Explicit and Implicit Requirements

1. *Explicit requirement*
   - A requirement that is documented on the Software Requirements Specification (SRS)
   - Includes both functional and non-functional requirements (quality attributes)

2. *Implicit requirement*
   - A requirement not documented in the SRS but is still expected in the application domain
   - E.g., Databases shall never store passwords in plain text form
   - E.g., Flight software shall never have a single point of failure

- Even if software does not violate SRS, if it violates implicit requirements
  → Still a defect!

Case 1: Is this a Defect?

- Observed behavior: Program loses data on system power loss.
  - Suppose requirements didn't specify behavior on power loss explicitly.
- Is this a defect?

- If app domain is a database: defect
  - Implicit requirement: committed transactions shall not lose data on power loss.
- If app domain is a game of solitaire: no defect, possibly an enhancement
  - No expectation that game will be saved on a power loss.

Case 2: Is this a Defect?

- Observed behavior: Program becomes unresponsive for 10 seconds.
  - Suppose requirements didn't specify performance expectations explicitly.
- Is this a defect?

- If app domain is a real-time game: defect
  - Implicit requirement: a real-time game must be responsive at all times.
- If app domain is a batch file copy tool: no defect, possibly an enhancement
  - No expectation that app will be fully responsive while the copy is happening.

☛ The answer depends on the application domain!

Understand Implicit Requirements

- Understand implicit requirements that come with application domain
  - You may need to do some research on prior literature on the subject matter
  - You may need to talk to a subject matter expert (SME) if you don't understand
  - Sometimes, the best SME is your customer
- Communication, Communication, Communication!

- Make implicit requirements explicit in the SRS once discovered
  - Prevents misunderstandings down the road

Reporting Defects

A Typical Bug Report Template

- SUMMARY
- DESCRIPTION
- REPRODUCTION STEPS
- EXPECTED BEHAVIOR
- OBSERVED BEHAVIOR
- SEVERITY (Optional)
- PRIORITY (Optional)

Summary - *succinct description of problem*

- A one sentence description of bug

- Examples:
  - Number of items in cart not refreshed when removing 2 items
  - CPU pegs at 100% after the addition of two nodes to the list
  - Title does not display after clicking "Next“

DESCRIPTION - *details of problem*

- A detailed description of everything the tester discovered
- Examples:
  - *Summary*: Number of items not refreshed when removing 2 items
  - *Description*: Removing 3, 4, and 5 items at once resulted in the same defective behavior. However, the number updated correctly if items are removed one at a time.

- Don’t overgeneralize: describing issue precisely helps developer
- Don’t merge multiple defects into one report
  - Removing 3, 4, 5 items mentioned only because likely caused by same defect

REPRODUCTION STEPS
- *Preconditions + Steps to Reproduce Defect*

- First, list *preconditions*
  - Omitting preconditions is most common cause of unreproducible bugs
    (Uncaught defects tend to occur only under specific conditions)
  - It's better to err on the side of over-specifying
    (If defect cannot be reproduced, it is very hard to fix)

- Next, enumerate *steps* that reproduce the defect precisely
  - Preferably a minimal set of steps leads to better reproducibility

REPRODUCTION STEPS

- BAD: Put some items in the shopping cart. Remove two items.

- GOOD:

  Preconditions:

  - OS: Windows 11
  - Browser: Chrome Version 151.0.7922.174
  - Site build: 4.2.1-rc3
  - Shopping cart is empty

  1. Launch and open https://my.shopping.site in browser.
  2. Add items A, B, and C to cart one by one, in that order.
  3. Remove items A and B from shopping cart at once.

EXPECTED AND OBSERVED BEHAVIOR

- *EXPECTED BEHAVIOR*: What you expected according to requirements.
  - Why is it important that this is part of the defect report?
    ☛ Describing expectations tells why observed behavior is deemed defective

- *OBSERVED BEHAVIOR*: What you ACTUALLY saw.
  - Dev team may not even be able to reproduce the bug
    - Dev team may not be able to reproduce your runtime environment that triggered the bug
    - Bug may be a heisenbug that only pops up once every few years
  - Attach a **screenshot** or verbatim **copy-and-paste** of what you saw
    - Much easier to locate the defect compared to a description of the failure

Why Expected Behavior is Needed

- Suppose this observed behavior was reported without expected behavior
  - Observed Behavior: 1 item(s) in cart is displayed.

- Can you figure out what the defect is?
  - It could be that the numerical value 1 is wrong.
  - It could be that the background color red is wrong.
  - It could be that the wording “item(s) in cart” is wrong.

Why Expected Behavior is Needed

- Adding the expected behavior surfaces the (perceived) defects.
  - Expected Behavior: The result is displayed on blue background.
  - Observed Behavior: 1 item(s) in cart is displayed.

- What are the (perceived) defects?
  - ~~It could be that the numerical value 1 is wrong.~~ ✗
  - It could be that the background color red is wrong. ✓
  - ~~It could be that the wording "item(s) in cart" is wrong.~~ ✗

- Perceived, because user supplied expected behavior may be inaccurate
  - Perhaps there is no requirement for background color → marked invalid defect

Tracking Defects

Tracking Defects

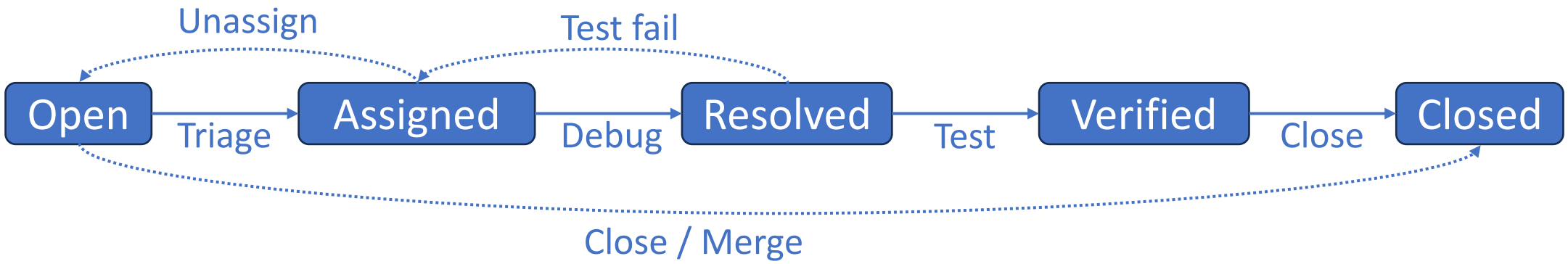
- Once defects are reported they need to be tracked
  - To make sure that they are fixed in a timely manner
  - To verify the fix corrects the defect and doesn't cause regression

- Must be done in a systematic way
  - Often hundreds of bugs at various stages of resolution
  - Often done with the help of a *bug tracking system*

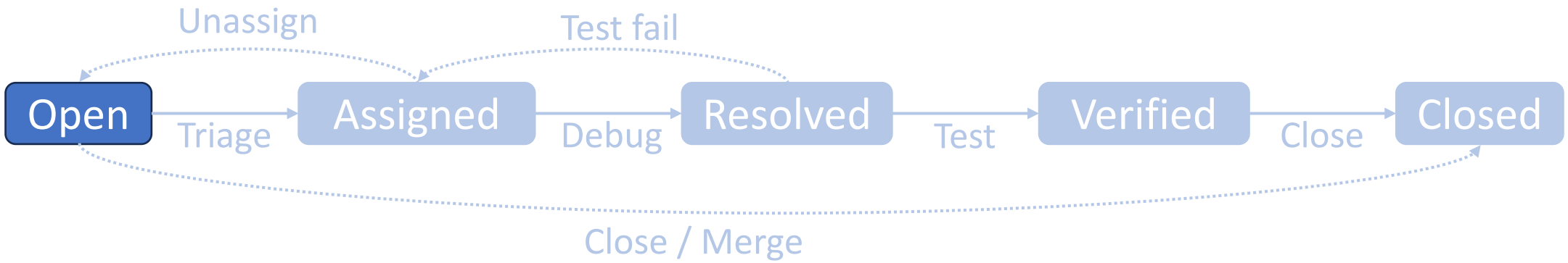
Tracking Defects

- In order to track, defects should have the following info:
  - Identifier: Typically, automatically assigned ticket number
  - Status: Stage of resolution ticket is in
  - Assignee: Developer who is in charge of resolution
  - Source: Associated test case, if applicable
  - Version found: commit version defect was found
  - Version fixed: commit version defect was fixed

Lifecycle of a defect

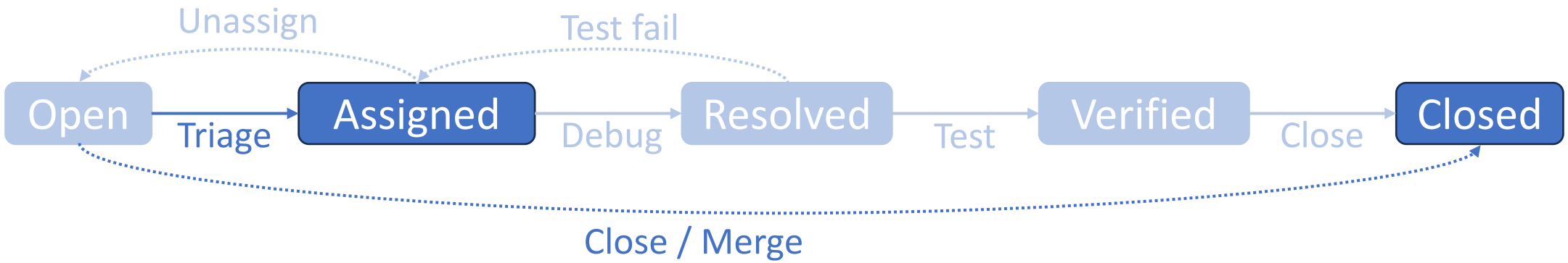


Lifecycle of a defect



- Defect ticket is first entered into the defect tracking system
  - Either by software QA team or end-user

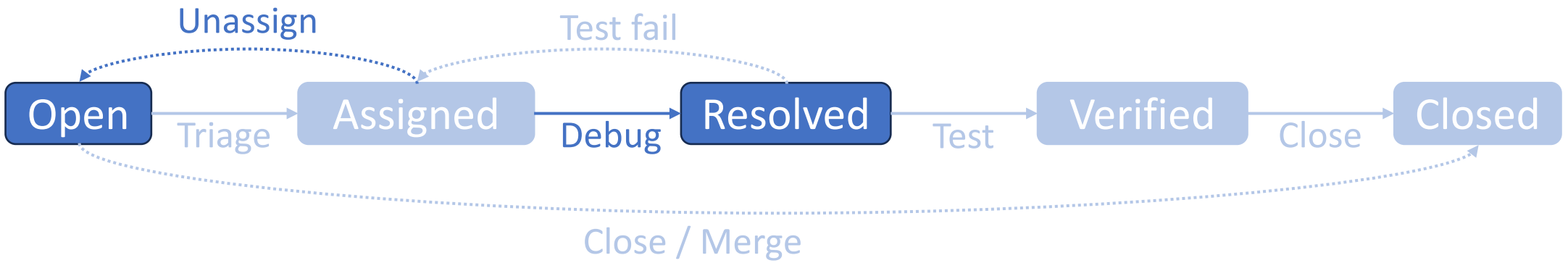
Lifecycle of a defect



- **Triage**: stakeholders (Dev lead, QA lead, PM, etc.) meet to determine
  1. Severity (impact on system operation) and priority (business urgency)
  2. Assignment of defect to a particular developer

- Triage may result in immediately closing the defect ticket
  1. If the defect is not a valid defect, simply close the ticket
  2. If the defect is a duplicate, merge with duplicate ticket

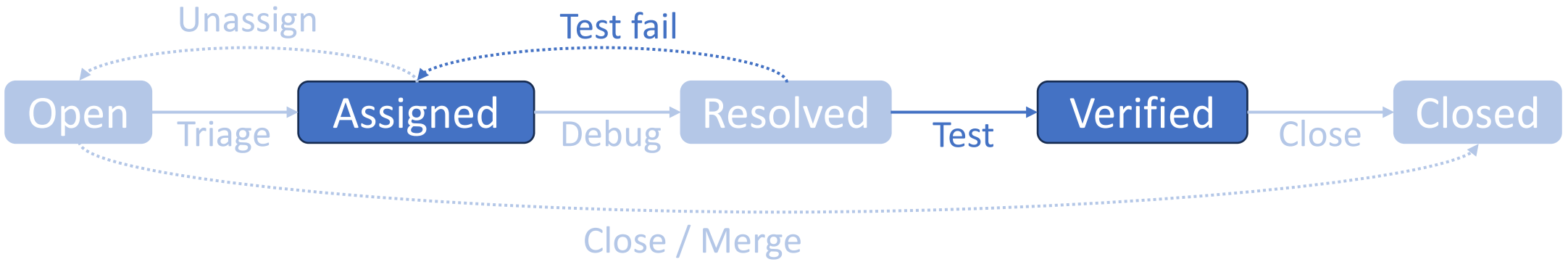
Lifecycle of a defect



- Assigned developer debugs the defect

- If developer cannot fix the defect, unassign and go back to triage

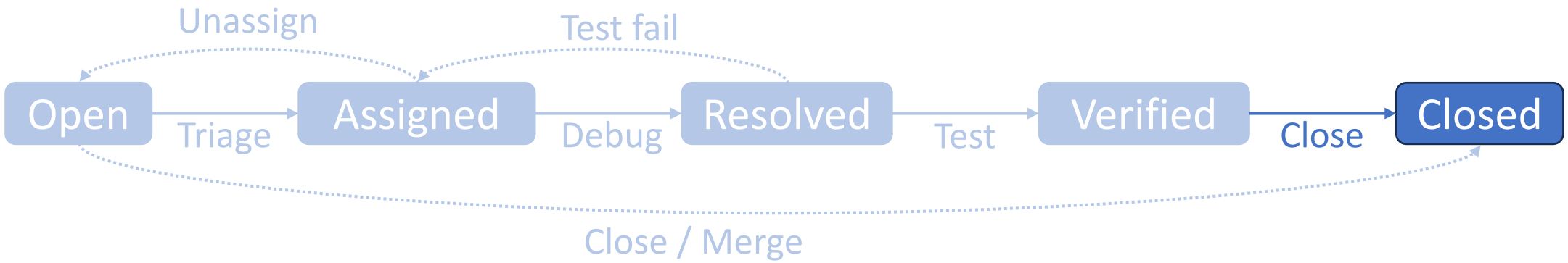
Lifecycle of a defect



- Software QA team performs various tests to verify fix
  - Regression test, Performance test, Security test, etc.

- If any test fails, reassign ticket back to developer

Lifecycle of a defect



- Close the defect ticket after verification is done
- Closed ticket remains in the system for future reference
- Add test case for defect in regression test suite

Make the defect lifecycle transparent to users

- It is normal for non-trivial software to ship with defects
  - Open-source projects regularly have hundreds of open issues at any given point
- Defects should be advertised, not hidden
  - Gives users confidence that defects are getting addressed
  - Allows users to work around defects
  - Allows users to participate in defect resolution
- … Except for security vulnerabilities
  - Which should be advertised only to software vendors who will fix them
  - Must be temporarily hidden from general public until they get fixed

Now Please Read Textbook Chapter 9